

Browser Compatibility Checker

Developer Documentation

Table of Contents

1. Overview
2. Installation
3. Usage
4. API Reference
5. Browser Support Matrix
6. Customization
7. Examples
8. Troubleshooting

Overview

The BrowserCompatibilityChecker is a JavaScript ES6 module that provides comprehensive browser and operating system detection and compatibility checking. It helps developers ensure their web applications are running in supported and allowed browser environments.

Key Features

- Detailed browser detection and version checking
- Operating system detection and version validation
- Configurable browser allowance rules
- Comprehensive compatibility reporting
- ES6 module support

Installation

```
// Import the module  
import BrowserCompatibilityChecker from './BrowserCompatibilityChecker.js';
```

Usage

Basic Usage

```
// Create an instance  
const checker = new BrowserCompatibilityChecker();  
  
// Check compatibility  
const result = checker.checkCompatibility();  
  
// Handle the result  
if (result.isCompatible) {  
    // Continue with application
```

```

} else {
    // Show incompatibility message
}

```

Advanced Usage

```

const checker = new BrowserCompatibilityChecker();
const result = checker.checkCompatibility();

// Detailed compatibility checking
if (!result.compatibilityDetails.browserAllowed) {
    console.log('This browser is not allowed in the current environment');
} else if (!result.compatibilityDetails.browserVersionSupported) {
    console.log('Browser version is not supported');
}

// Access specific browser information
console.log(`Browser: ${result.browser} v${result.version}`);
console.log(`Operating System: ${result.os} ${result.osVersion}`);

```

API Reference

Class: BrowserCompatibilityChecker

Constructor

```
const checker = new BrowserCompatibilityChecker();
```

Methods

checkCompatibility() Returns an object containing all browser compatibility information.

Returns:

```

{
    // Browser identification flags
    isChrome: boolean,
    isFirefox: boolean,
    isSafari: boolean,
    isEdge: boolean,
    isEdgeChromium: boolean,
    isOpera: boolean,
    isBrave: boolean,
    isIE: boolean,

    // Browser information
    browser: string,

```

```

    version: string,
    majorVersion: number,
    isAllowed: boolean,

    // OS information
    os: string,
    osVersion: string,

    // Compatibility results
    isCompatible: boolean,
    compatibilityDetails: {
        osSupported: boolean,
        browserSupported: boolean,
        osVersionSupported: boolean,
        browserVersionSupported: boolean,
        browserAllowed: boolean
    },

    timestamp: string
}

```

detectBrowsers() Internal method that performs browser detection.

detectOS() Internal method that performs operating system detection.

compareVersions(v1: string, v2: string) Internal method for comparing version strings.

Browser Support Matrix

Windows

Browser	Minimum Version	Maximum Version	Allowed
Chrome	72	∞	Yes
Firefox	66	∞	Yes
Microsoft Edge	80	∞	Yes
Internet Explorer	11	11	No
Opera	60	∞	No
Brave	1	∞	No

macOS

Browser	Minimum Version	Maximum Version	Allowed
Chrome	72	∞	Yes
Firefox	66	∞	Yes
Safari	14.1	∞	Yes
Opera	60	∞	No
Brave	1	∞	No

Linux

Browser	Minimum Version	Maximum Version	Allowed
Chrome	72	∞	Yes
Firefox	66	∞	Yes
Opera	60	∞	No
Brave	1	∞	No

Customization

Modifying Browser Requirements

You can modify the BROWSER_REQUIREMENTS object in the constructor to adjust:
 - Supported browsers - Version requirements - Browser allowance - OS support

Example:

```
class CustomBrowserChecker extends BrowserCompatibilityChecker {
  constructor() {
    super();
    // Modify Chrome requirements
    this.BROWSER_REQUIREMENTS.Windows.browsers.Chrome = {
      min: 80,
      max: 100,
      allowed: true
    };
  }
}
```

Examples

Basic Compatibility Check

```
import BrowserCompatibilityChecker from './BrowserCompatibilityChecker.js';

function checkBrowserSupport() {
  const checker = new BrowserCompatibilityChecker();
  const result = checker.checkCompatibility();
}
```

```

    if (!result.isCompatible) {
      if (!result.compatibilityDetails.browserAllowed) {
        showUnsupportedBrowserMessage(
          `${result.browser} is not allowed in this application.`
        );
      } else if (!result.compatibilityDetails.browserVersionSupported) {
        showUpdateBrowserMessage(
          `Please update your ${result.browser} to version ${
            result.BROWSER_REQUIREMENTS[result.os].browsers[result.browser].min
          } or higher.`
        );
      }
    }

    return result.isCompatible;
  }
}

```

Custom Error Handling

```

try {
  const checker = new BrowserCompatibilityChecker();
  const result = checker.checkCompatibility();

  if (result.error) {
    console.error('Compatibility check failed:', result.error);
    // Handle error case
    return;
  }

  // Process result
} catch (err) {
  console.error('Unexpected error during compatibility check:', err);
}

```

Troubleshooting

Common Issues

1. **Browser Not Detected Correctly**
 - Ensure user agent string is accessible
 - Check if browser detection method is up to date
 - Verify browser version format parsing
2. **False Incompatibility Results**
 - Verify BROWSER_REQUIREMENTS configuration
 - Check version comparison logic
 - Confirm OS detection accuracy

3. Module Import Issues

- Ensure proper ES6 module support
- Check file path and extension
- Verify server MIME types configuration

Debug Mode

To enable detailed logging:

```
const checker = new BrowserCompatibilityChecker();  
const result = checker.checkCompatibility();  
console.log('Detailed compatibility result:', JSON.stringify(result, null, 2));
```

For additional support or to report issues, please contact the development team.